



REINFORCEMENT LEARNING

AULA 6

Prof. Gian Carlo Brustolin



CONVERSA INICIAL

Sabemos que, ao enfrentarmos problemas complexos de IA, um dos principais paradigmas que definem o sucesso do projeto é a extração de parâmetros para a composição do *dataset*. É necessário transformar o problema real em um protótipo matemático tratável pelas modelagens que estudamos.

As técnicas de DL resolveram parcialmente esse problema, pelo uso de redes convolucionais, que são capazes de extrair autonomamente parâmetros do espaço de entrada. A aplicação mais conhecida dessas redes se dá no tratamento de imagens. As CNNs amostraram a imagem, aplicando filtros neurais. Com base nessa filtragem criam-se mapas de características (*feature maps*) da imagem. Os mapas podem sofrer novas filtrações convolucionais, aprofundando o processo de extração.

Nesta etapa vamos estudar a aprendizagem por reforço profundo (DRL, *deep reinforcenment learning*), que é o resultado da inserção estável de algoritmos neurais de *deep learning* em métodos de aprendizagem por reforço. Entre os algoritmos híbridos de DRL mais conhecidos, enumeramos DQN (Deep Q-Network), DQN duplo, TRPO (Trust Region Policy Optimization) e PPO (Proximal Policy Optimization). Por se utilizarem de algoritmos predominantemente neurais, de característica naturalmente genérica, algumas implementações de biblioteca começam a surgir, facilitando sua utilização pelos programadores. Unity.com é um exemplo a ser explorado.

Apesar disso, o tema DRL é bastante novo na literatura e há muito a se caminhar para se estabelecer algoritmos ótimos, estáveis e confiáveis de reforço profundo, o que torna nosso presente estudo especialmente interessante.

TEMA 1 – PROBLEMAS PARA USO DE RNAs EM REINFORCEMENT LEARNING (RL)

Como já descobrimos em conteúdos anteriores, implementações de RL por métodos simbólicos dependem fortemente do desenvolvimento de algoritmos proprietários. Verificamos também que a dimensionalidade dos problemas, para serem enfrentados pela codificação de métodos matemáticos, é limitada. A alta complexidade dos algoritmos exige uma adequação do problema às modelagens. Essa adequação requer significativo esforço



computacional que, por vezes, supera o da própria solução do problema.

Redes neurais artificiais são uma boa aproximação para cálculos mais genéricos de algoritmos de aprendizagem por reforço. As primeiras tentativas nesse sentido, entretanto, conservaram a “maldição da dimensionalidade” proveniente das soluções simbólicas. A evolução das técnicas conexionistas profundas, que ocorreu concomitantemente às tentativas de aplicação de redes neurais em RL, finalmente suavizou a maldição. Vamos entender essas tentativas iniciais de implementação.

1.1 O problema da instabilidade da estimativa neural de $Q(s,a)$

A semelhança entre os processos iterativos de solução do MDP e das equações de Bellman conduziu as pesquisas no sentido de adequar o treinamento de uma rede neural para que predissesse os valores de Q .

Redes neurais são treinadas com base em coleções numerosas de dados. Essas coleções são tão vastas que nos permitimos amostrar apenas uma pequena parte delas para compor os padrões de treinamento. Essa profusão de dados torna os padrões estatisticamente estáveis.

Os dados disponíveis em RL são, todavia, bastante particulares. Como já estudamos, as informações que o agente tem do meio são ruidosas, incompletas e esparsas, disponíveis somente após a interação temporal com o meio. Outra forte característica de problemas markovianos é a grande sensibilidade a variações de Q . Uma pequena alteração em Q pode indicar uma mudança de política, ou seja, é capaz de alterar toda a distribuição de dados. Uma coleção com essas características rompe com as premissas de estabilidade estatística dos padrões de treinamento para uma RNA.

As técnicas temporais de RL incorporam, ainda, a necessidade de atualizar os parâmetros da rede neural dinamicamente, ou seja, é preciso que a rede permaneça em treinamento constantemente e realize as previsões durante o treinamento. Essa parece ser a fórmula perfeita para um sistema instável e divergente.

Por outro lado, tanto as RNAs quanto as técnicas de RL têm sua origem na observação da fisiologia e do comportamento animal. Analisar um ambiente ruidoso de alta dimensionalidade, utilizando informações multidimensionais para generalizar as experiências do passado de forma a escolher a melhor ação



presente, é o cotidiano de sucesso tanto de humanos quanto de animais (Mnih et al., 2015, p. 529). De fato, a aprendizagem por reforço, suportada por uma rede neural complexa, constitui a maior parte da aprendizagem humana.

Dessa forma a estabilidade dos algoritmos de DRL precisa ser possível. O enfrentamento tradicional é feito por treinamento recorrente. A cada nova amostra, a rede é retreinada, até que o novo padrão seja assimilado. Como as variações de Q tendem a ser pequenas, conforme nos aproximamos de Q^* , devemos reduzir α . Com velocidade de aprendizagem mais baixa, valores típicos do número de épocas para a convergência são assustadoramente altos, em torno de alguns milhares (Riedmiller, 2005, p. 319). Essa conclusão inviabilizaria o uso de RNAs em RL, uma vez que o agente precisa responder ao meio de forma *online*. Dito de outra forma, o problema é solucionável, mas não é computável por essa abordagem.

Não nos desesperemos ainda. Temos mais problemas a enfrentar.

1.2 O problema da perda de memória

As primeiras tentativas de uso de RNAs para tratamento de problemas de RL apresentaram vários inconvenientes, além da questão da instabilidade e da factibilidade computacional.

A questão da memória global da RNA é outro exemplo aterrorizante. A memória de uma rede neural (M) permite a generalização da aprendizagem em ações futuras e isso é uma característica interessante, que faculta o tratamento de problemas de ordem de Markov alta. Porém, quando carregamos a memória da rede com padrões, presume-se que M se manterá intacta durante a operação. Essa presunção não é verdadeira para RL. Aprender por reforço significa alterar M temporalmente, de forma a refletir novas recompensas obtidas do meio.

Uma pequena alteração em M modifica virtualmente todas as regiões de memória, destruindo esforços de memorização anteriores. O enfrentamento desse problema normalmente prolonga indefinidamente o treinamento e, eventualmente, o faz falhar.

Neural Fitted Q Iteration (NFQ) é um algoritmo que busca restaurar o conhecimento prévio sempre que uma atualização é realizada. A implementação parte da ideia de memorizar as transições de estado anteriores reimplantando-as a cada *update* (Riedmiller, 2005, p. 318).



Para concretizar essa ideia, seu criador, Riedmiller, precisaria de um algoritmo de treinamento mais rápido que o tradicional BP. Ele desenvolve então a propagação resiliente (R-prop), que utiliza a derivação do erro em relação a cada peso sináptico individualmente, enquanto o BP clássico considera a derivada de todo o vetor W como gradiente de redução, ou seja:

$$\Delta w = - \partial C / \partial w$$

Dessa forma, tomando-se w_{ij} como o peso que conecta o neurônio “I” com o neurônio “J” podemos escrever para NFQ (Igel; Husken, 2000, p. 116):

$$\Delta_{ij}^{(t)} := \begin{cases} \eta^+ \cdot \Delta_{ij}^{(t-1)}, & \text{if } \frac{\partial E^{(t-1)}}{\partial w_{ij}} \cdot \frac{\partial E^{(t)}}{\partial w_{ij}} > 0 \\ \eta^- \cdot \Delta_{ij}^{(t-1)}, & \text{if } \frac{\partial E^{(t-1)}}{\partial w_{ij}} \cdot \frac{\partial E^{(t)}}{\partial w_{ij}} < 0 \\ \Delta_{ij}^{(t-1)}, & \text{else,} \end{cases}$$

O algoritmo decide a direção de atualização de cada w_{ij} individualmente. O cálculo do valor de Δw também segue uma regra inusitada. Escolhe-se para ele um valor inicial a partir da derivação da função de ativação, como em BP, mas esse valor será alterado de forma incremental nas próximas iterações. Se, em mais de uma propagação, a direção do gradiente se mantiver constante, Δw será incrementado, caso contrário, depreciado. Esse método é algumas vezes mais rápido para se aproximar do gradiente nulo de erro do que o BP tradicional (Igel; Husken, 2000, p. 120).

O algoritmo da Figura 1 ilustra como NFQ pode ser implementado com o uso do treinamento por R-prop. Na figura, $c(s,u,s')$ denota a recompensa de transição do estado s para s' pela ação u .



Figura 1 – Loop NFQ

```
NFQ_main() {  
  input: a set of transition samples  $D$ ; output: Q-value function  $Q_N$   
   $k=0$   
  init_MLP()  $\rightarrow Q_0$ ;  
  Do {  
    generate_pattern_set  $P = \{(input^l, target^l), l = 1, \dots, \#D\}$  where:  
       $input^l = s^l, u^l$ ,  
       $target^l = c(s^l, u^l, s'^l) + \gamma \min_b Q_k(s'^l, b)$   
    Rprop_training( $P$ )  $\rightarrow Q_{k+1}$   
     $k := k+1$   
  } WHILE ( $k < N$ )
```

Fonte: Riedmiller, 2005, p. 319.

Observe que o NFQ presume um treinamento pelo uso de uma batelada de padrões $X=\{s,u,s'\}$. Assim, NFQ propõe a criação de uma rede de treinamento P atualizada a cada conjunto selecionável de amostras X e posta em operação em seguida. Essa solução contorna o problema da computabilidade por dois caminhos: graças à velocidade alta de iterações com o uso de R-prop, a rede P pode estar apta a decidir quando requisitada, mas, se não estiver, há uma rede de resposta não ótima disponível.

O problema parecia estar resolvido. A questão que não foi tratada por Riedmiller, entretanto, é o aprofundamento das redes. Os agentes de IA foram inicialmente treinados para operar em ambientes modelados matematicamente. Conforme os ambientes se aproximam da realidade, os agentes demandam redes mais profundas, capazes de mapear o meio e interpretá-lo.

A necessidade de treino, em uma rede independente, a cada batelada de padrões novos, não é um problema para RNAs com poucas camadas. Em aplicações que exijam redes profundas NFQ não é suficientemente rápido e passa a impedir a escolha da ação de maior valor (Q_{max}), em tempo aceitável, para a maioria dos ambientes de teste. O agente responderá, segundo o treinamento não ótimo, por tempo inaceitável para sua sobrevivência no ambiente. Voltamos ao problema da computabilidade que achávamos ter resolvido.



1.3 Jogando Atari

Mnih et al., observando o comportamento animal, partiu do pressuposto que uma rede neural complexa é capaz de tratar RL em ambientes reais complexos (Mnih et al., 2015). Com base nessa convicção, criou, em 2014, o primeiro algoritmo conexionista estável e computável para aprendizagem por reforço, dito DQN (Deep Q-network Agent). Esse agente aprendeu diretamente de um ambiente multidimensional complexo, criando políticas bem-sucedidas de ação. O agente foi testado como um jogador humano no clássico jogo Atari 2600.

De forma a simular o jogador convenientemente, Mnih forneceu ao agente, como entradas, apenas os pixels da tela de jogo e o score atual. Após 49 partidas o agente atingiu *performance* semelhante à de jogadores humanos, sedimentando a estabilidade de algoritmos de DRL em ambiente de alta dimensionalidade. Em nosso próximo tópico vamos entender DQN e como os problemas de instabilidade e computabilidade foram enfrentados. Também vamos compreender como Mnih resolveu os problemas que nos assolam neste momento.

TEMA 2 – APRENDIZAGEM EM REDES Q PROFUNDAS

Em nosso tópico anterior entendemos os paradigmas enfrentados quando desejamos utilizar IA conexionista para resolver a aprendizagem de agentes RL. Sabemos que o algoritmo neural profundo de cálculo de Q (DQN) obteve sucesso na solução desses problemas. Vamos agora detalhá-lo.

2.1 Fundamentos de DQN

As redes neurais artificiais adquirem memória pelo ajuste dos parâmetros livres. Normalmente esse ajuste é feito para que se minimize o erro de previsão da saída em função da entrada. Quando uma RNA é treinada para uso em DRL o que se objetiva é o reconhecimento do estado do agente e o ajuste dos pesos para que a saída da rede informe a ação que resultará em melhores recompensas. O problema que restava insolúvel era a computabilidade do algoritmo estável NFQ, cuja ideia de criação de uma rede P de treinamento “a frio” obteve sucesso para redes de baixa profundidade. Mnih dividiu o



processamento do problema com o uso de uma CNN, deixando a fase de cálculo de Q isolada da extração de parâmetros do meio.

Na clássica aplicação em *video game* uma CNN é utilizada para reconhecer o estado com base na interpretação da imagem do console e, em seguida, nas camadas de classificação, escolherá a ação de maior Q. Essa CNN não terá camada de *pooling*. Como já sabemos, a função de *pooling* extrai invariâncias, tornando a imagem independente, por exemplo, de posicionamento. Para a análise de RL que desejamos, a posição de dado objeto em relação ao meio não pode ser desprezada. Após a convolução a imagem é extraída e passamos ao cálculo de Q. Observe que a convolução da imagem traduz a imagem como simples estados, reduzindo a dimensionalidade do problema de escolha da ação ótima.

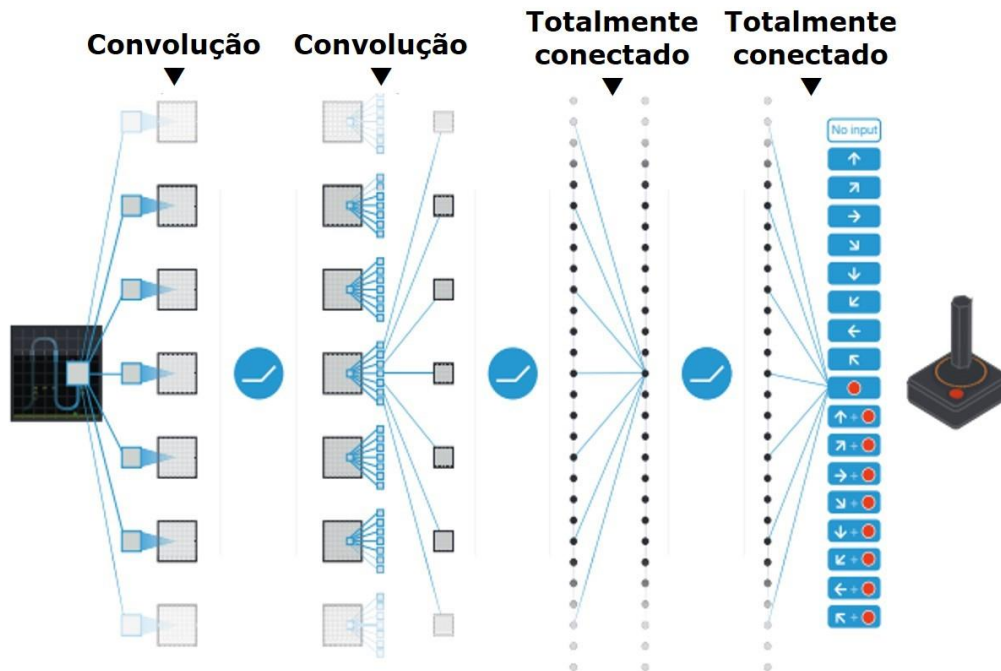
Essa escolha será realizada por uma MLP ReLU (*fully connected ReLU network*) como se vê na Figura 2, na qual cada neurônio de saída corresponderá a uma ação do jogo. Embora a decisão de qual ação executar possa ser igualmente tomada por algoritmos implementados de forma simbólica, o controle e a manutenção das tabelas Q tornam o processamento lento e de alto custo. Essa incapacidade computacional é diagnosticada quando problemas com milhares de estados são enfrentados por meios não conexionistas.

No algoritmo DQN a MLP é treinada para que sua memória aproxime a função Q. Dessa forma, ao receber, após o treinamento inicial, um estado V de entrada, a rede escolherá o Q de valor mais alto para V.

A arquitetura proposta para o Atari está ilustrada na figura a seguir.



Figura 2 – Arquitetura da DQN para o *video game*



Crédito: Jefferson Schnaider.

Devemos entender ainda como o algoritmo equaciona os problemas clássicos de estabilidade e memória. Nmit propõe duas implementações para contornar esses problemas: *target network* e *experience replay*. Vamos então conhecê-las.

2.2 Target network

O problema de estabilidade da estimativa de valor foi contornado pela criação de uma rede de treinamento dita TN (*target network*). Sabemos que pequenas alterações no valor de Q podem ter consequências importantes para a mudança da política. O ruído próprio da aprendizagem neural bloqueia boa parte da sensibilidade da rede a essas pequenas variações. Para que entendamos o processo vamos dividir o ciclo de treinamento da rede em intervalos discretos de “C” eventos de duração total “t”.

No momento t1, o algoritmo de DQN toma as decisões baseado em uma rede de operação estável, mas armazena os resultados de “C” ações. Em $t_2 = t_1 + t$ o algoritmo troca a rede de operação, substituindo-a pela rede de treinamento (TN), cuja aprendizagem foi feita, durante t1, com os “C” eventos



anteriores obtidos em $t1-t$.

Durante $t2$ um novo ciclo de memorização de “C” eventos se inicia, enquanto, paralelamente, uma nova rede TN é treinada conforme os “C” eventos memorizados em $t1$.

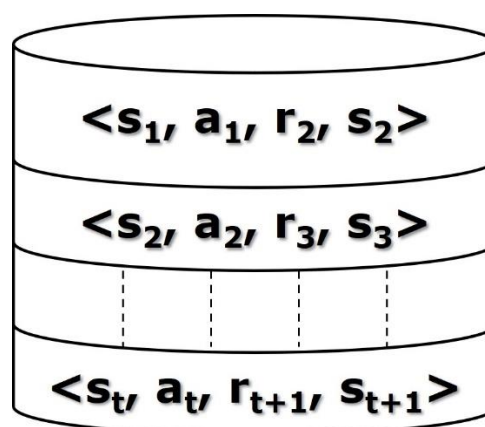
Com esse subterfúgio de treinar redes diferentes a “C” ações impede-se o enviesamento do treinamento, principalmente nas proximidades de Q^* , resolvendo um dos problemas clássicos provocados pelo uso de redes neurais na estimativa de Q . Resta ainda um problema a enfrentar, o qual estudaremos a seguir.

2.3 Experience replay

Um segundo problema de implementação de algoritmos conexionistas para RL é o fenômeno do VGP anteriormente apresentado, que provoca o desvanecimento da memória de episódios passados. Retomando o exemplo de um *video game*, o mero conhecimento do estado atual de um ator do jogo não permite prever em que direção, por exemplo, esse ator está se movendo. Dessa forma enfrentamos um MDP de ordem superior e o problema de VGP impedirá o sucesso do agente.

DQN enfrenta esse problema pela criação de uma memória episódica externa à rede. As experiências do agente são definidas pelo vetor linear $e_t = (s_t, a_t, r_t, s_{t+1})$ e a memória $D_t = \{e_1, \dots, e_t\}$, conforme se vê na Figura 3.

Figura 3 – Memória episódica DQN



A função de valor para DQN é $Q(s, a, \Theta)$, onde Θ representa os parâmetros livres da RNA, ou seja, pesos sinápticos e *bias*. O *dataset* D é randomicamente



disposto, ou seja, quando obtido um novo Q, este é acrescido randomicamente a essa coleção. A função de custo é calculada como a distância quadrática entre os Qs como abaixo.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

Nessa equação Θ_i representa os novos parâmetros da rede ajustados a partir dos atuais Θ_i^- . A atualização de Q ocorre somente após “C” ações do agente e é realizada por BP, minimizando a função de custo descrita acima. Devemos recordar que o treinamento é realizado na rede “fria” TN, ou seja, em uma rede que não está operando, conforme descrito no item anterior.

Esse procedimento provou contornar o problema de VGP mesmo com muitos ciclos de treinamento com o uso de gradiente descendente.

TEMA 3 – PPO – PROXIMAL POLICY OPTIMIZATION

Algoritmos de PPO são considerados o estado da arte em DRL em termos de *performance*. O comportamento das alterações internas ao algoritmo, entretanto, não é plenamente conhecido. Esse algoritmo de treinamento é uma evolução do TRPO (Trust Region Policy Optimization).

A ideia básica do criador, Schulman et al. (2017), é alterar a forma como os pesos neurais são atualizados, disparando pequenas bateladas de treinamento a cada novo estado de forma análoga ao NFQ e ao DQN. No algoritmo PPO se pretende, entretanto, atualizar a política do agente com interações despadronizadas. Vamos descrever brevemente a TRPO para que possamos, posteriormente, melhor compreender o método PPO.

3.1 TRPO

O algoritmo de TRPO baseia a otimização de Q em algo diverso do controle do gradiente de erro. Observou-se que há uma relação entre o ganho de utilidade entre uma política π e outra $\tilde{\pi}$. Esse ganho, M_i , na equação a seguir, pode ser estimado pela divergência matemática “D” entre elas somada à estimativa local “L” da utilidade de $\tilde{\pi}$.

$$M_i(\pi) = L_{\pi_i}(\pi) + C \cdot D(\pi_i, \pi)$$

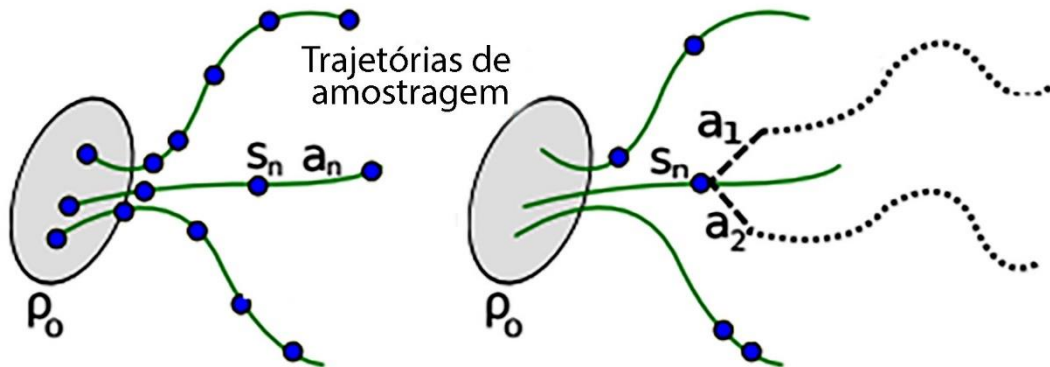
$$\text{onde } C = \frac{2\epsilon\gamma}{(1-\gamma)^2}$$

A estimativa local da utilidade $L(\pi)$, por sua vez, guarda relação com a utilidade $\eta(\pi)$ e a distribuição de probabilidade da visita a determinado estado $q(s)$, conforme mostrado a seguir, onde $A(s,a) = Q(s,a) - V(s,a)$.

$$L_{\pi}(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\pi}(s) \sum_a \tilde{\pi}(a|s) A_{\pi}(s,a).$$

Dessa forma fica claro que a estratégia de visita ao estado tem influência sobre o ganho de utilidade e, portanto, sobre a otimização da política. Pode-se então pensar em buscas em uma região em torno da trajetória ótima, conforme se vê na figura a seguir.

Figura 4 – Trajetórias de amostragem



Essa busca de entorno tem aplicação na otimização do processo de RL para movimentação de robôs, uma vez que modela eficientemente o comportamento estocástico do deslocamento em ambientes reais.

Finalmente a função $M_i(\pi)$ pode ser usada como uma função de custo substituta, uma vez que avalia indiretamente a eficiência da iteração. Quanto menor $M_i(\pi)$, mais próximos estaremos da estabilidade do gradiente de erro.

O estudo sobre a influência do caminharmento, ou escolha de estados a explorar, do TRPO, além de motivar sua evolução “grampeada”, PPO, forneceu a ideia geradora para outro algoritmo promissor, o Go-Explore, criado no



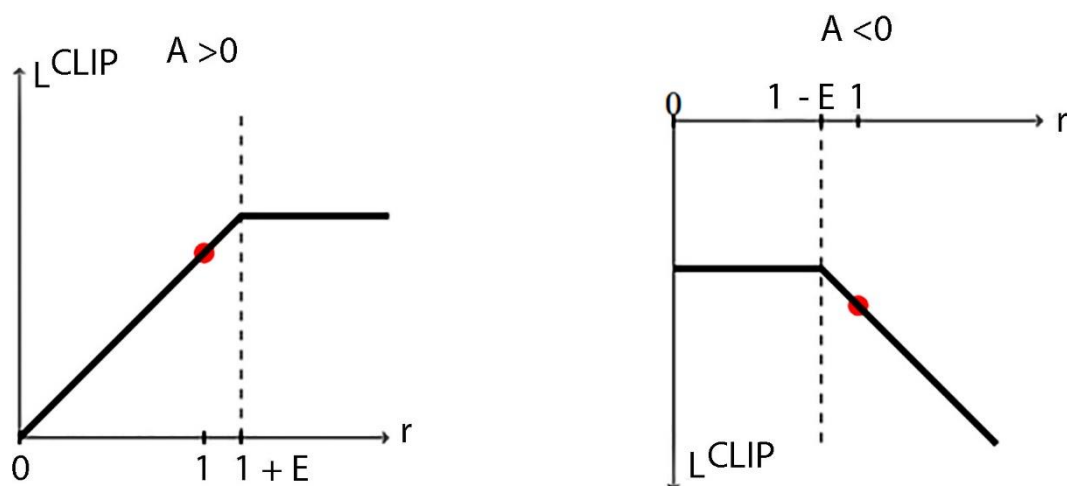
laboratório de IA do Uber por Ecoffet et al. (2021). O artigo inaugural e seu complemento, referenciado aqui, é uma leitura bastante recomendada.

3.2 PPO

Como comentamos, PPO baseia sua implementação nas conclusões obtidas no algoritmo TRPO, principalmente no que se refere ao caminhar das buscas. Na implementação do algoritmo de otimização proximal, intenta-se o uso de uma rede neural de camada única e função de ativação ReLU para a avaliação de Q , como em DQN. A grande diferença no método é a inserção de um limitador na variação da política, ou seja, embora calcule-se a atualização dos parâmetros de Markov, a variação entre os parâmetros antigos e os novos se vê limitada pelo “grampeamento” (*clipping*) dos valores.

O grampeamento limita a variação entre $1+\Theta$ e $1-\Theta$, onde Θ é um macro parâmetro, tal que $0<\Theta<1$. Essa limitação confere a necessária estabilidade ao modelo. Em ambientes estocásticos, ajustes abruptos na política tornam as decisões de ação de difícil controle, posto que a reação do meio não é plenamente conhecida. O resultado do grampeamento sobre a utilidade local (L) pode ser visto na figura a seguir.

Figura 5 – Grampeamento



Fonte: Schulman, 2017, p.3.

Além desse efeito de estabilidade das decisões, como o cálculo de parâmetros é feito por métodos neurais, é necessário conter a velocidade de



aprendizagem para evitar os efeitos negativos da não linearidade da superfície de erro. O pleno detalhamento desse algoritmo não será objeto deste estudo, mas o artigo de Schulman (2017), criador do método, referenciado em nossa bibliografia, fornece o aprofundamento necessário para aqueles que o desejarem.

TEMA 4 – APLICAÇÃO DE DRL EM OBJETOS IoT

Técnicas estáveis de DRP são bastante recentes na literatura científica. Como estudamos anteriormente, o algoritmo de DQN foi testado em 2015; a publicação da teoria de PPO, que acabamos de estudar, é de 2017.

Após a publicação do estudo inaugural a comunidade científica parte para uma série de tentativas de aplicação em ambientes reais. Essas tentativas ainda não consolidam os algoritmos como soluções viáveis. São necessárias várias aplicações semelhantes comparadas para que a consolidação ocorra.

Considerando o que acabamos de comentar, apesar da recente publicação, algumas aplicações de algoritmos de DRP já são consideradas soluções para problemas clássicos. Um desses campos de aplicação é o atendimento aos objetos IoT.

Objetos inteligentes conectados à internet, ou objetos IoT, não são uma novidade. As mães desses objetos, as redes de sensores e atuadores, são utilizadas em automação industrial há algumas décadas. A popularização do uso de objetos IoT em residências, cidades inteligentes e automação de redes elétricas é, todavia, recente.

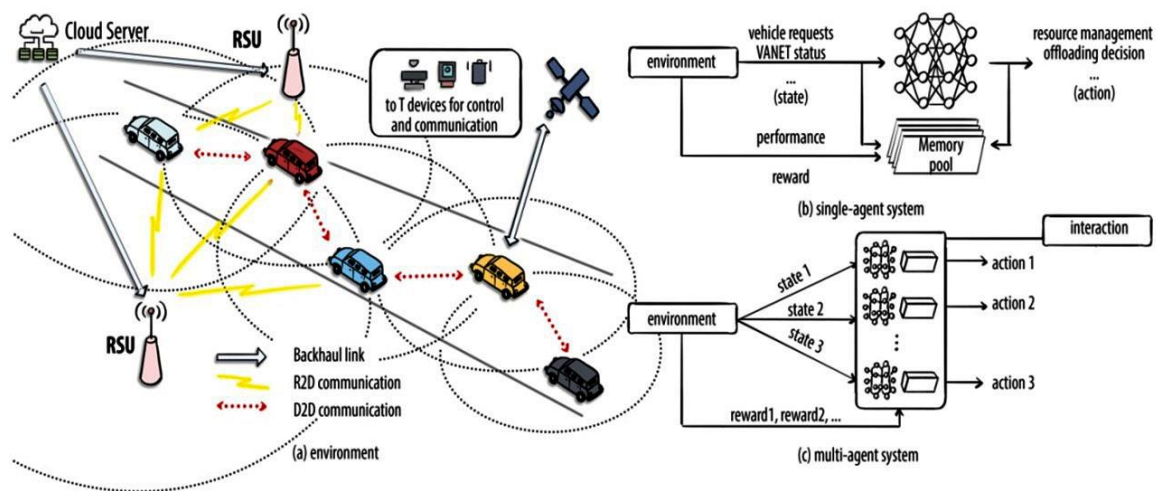
A profusão desses objetos, de alta adaptabilidade e baixo custo, gerou uma demanda inesperada por complexas estruturas de rede que permitam sua conectividade segura. Embora a produção de dados proveniente de um objeto IoT seja desprezível, a somatória de milhões desses pequenos objetos tem o potencial de saturar a capacidade de processamento das redes atuais. Soluções de *edge* e *fog computing* contornam esse problema, mas acrescentam mais complexidade à conectividade de IoT. Esta, aliás, já é, por si, suficientemente complexa, dada a heterogeneidade das soluções disponíveis, como WiSun, 5G, NB LTE, Wimax, WiFi 6 e LoRa, todas com protocolos distintos e provenientes de padronizações herméticas.

Uma das demandas mais densas por computação de borda e

conectividade se dá nas aplicações de objetos IoT em cidades inteligentes (*smart cities*). As técnicas de DRL são uma boa possibilidade de tratamento de dados na borda, facultando decisões rápidas e adaptativas para as demandas de dados sem o envolvimento do processamento pesado no *core* da rede.

A supervisão de veículos autônomos é um dos desafios, associados a cidades inteligentes, de difícil enfrentamento. A figura a seguir ilustra como uma arquitetura DRL, associada a multiconectividade, pode permitir as decisões em tempo real demandadas pela aplicação.

Figura 6 – DRL em *smart cities*



Crédito: Jefferson Schnaider.

Observe que a conectividade heterogênea, nessa aplicação de controle veicular, insere *delays* próprios na aquisição de dados pelo algoritmo de decisão. Nesse caso, um tratamento temporal dos dados é necessário. Aqui se pode ver o quanto a velocidade de processamento do algoritmo decisório está vinculada ao sucesso da aplicação. Soluções testadas em ambientes de jogos, com bom desempenho, podem falhar nas aplicações de tempo real se não contarem com algoritmos otimizados.

Guardadas as mesmas observações, a gestão de demanda em redes elétricas inteligentes é outra aplicação ideal para processos de RL. Nessa gestão o estado do meio está em constante alteração, refletindo o momento atual do sistema elétrico que alimenta determinada unidade consumidora. A decisão de uso de microgeração ou drenagem de corrente da rede deve ser tomada a cada



lapso temporal, em função do estado atual do sistema e da disponibilidade de carga do gerador local. Arquiteturas de RL por TD, computadas na borda da unidade, têm sido utilizadas com sucesso nessa decisão.

Se encararmos as redes de objetos IoT, mesmo aquelas de pequeno porte, a administração de segurança e a gestão da conectividade concomitante com as demais entidades de rede são atividades de alta complexidade, cujo entendimento e configuração dinâmica demandam soluções de IA adaptativas a exemplo dos algoritmos de DRL.

TEMA 5 – APLICAÇÃO DE DRL EM ROBÓTICA

Outra aplicação exitosa de algoritmos de RL e DRL ocorre em controle de autômatos e robôs. Os principais desafios desse casamento são o estado inicial de ignorância do robô e os métodos para evitar comportamentos indesejados aprendidos diretamente do meio (Ibarz et al., 2021, p. 699).

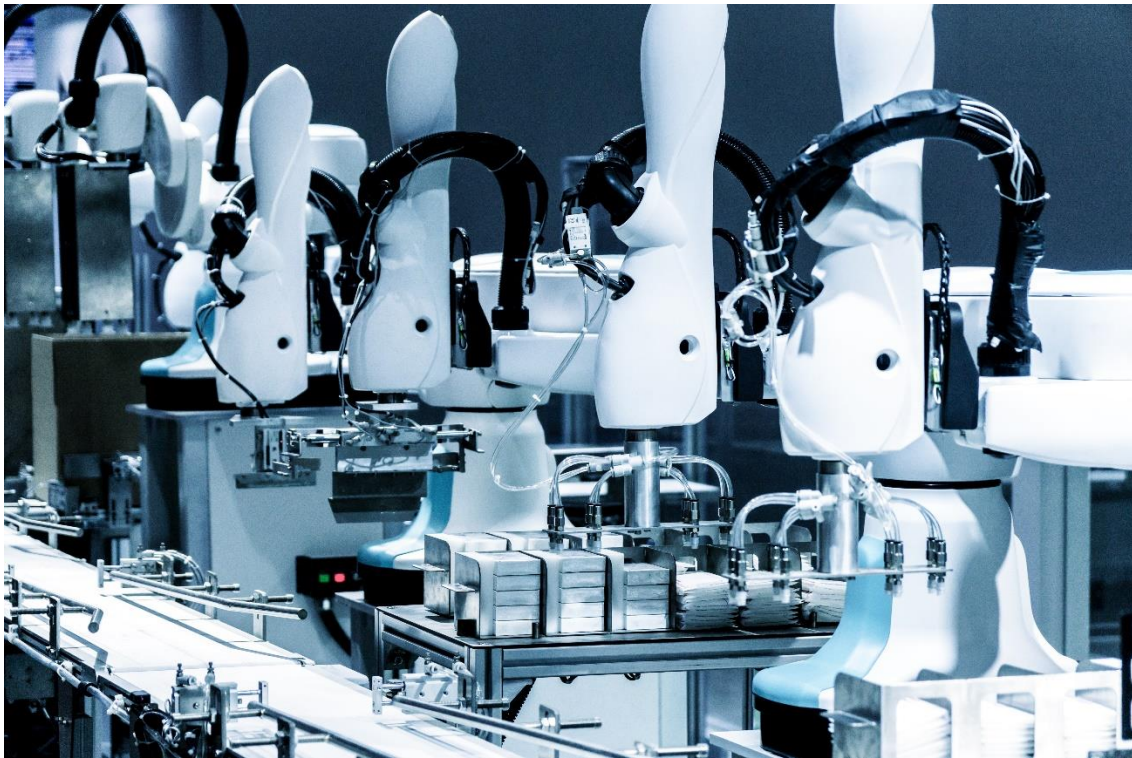
Alguns aprendizados robóticos são especialmente aderentes aos algoritmos de DRL. Citaremos alguns relevantes, seguindo o artigo de Ibarz et al. (2021).

O aprendizado de manipulação de ferramentas é um campo interessante de aplicação, uma vez que sistemas de DRL permitem o aprendizado de ciclo completo, sem necessidade de particularizações de algoritmo a cada nova tarefa a ser aprendida. Dessa forma, o autômato, com base na análise sensorial do meio e de análise da utilidade de cada ação, encontra a política correta após algumas interações com o meio.

Nesse caso, algoritmos que permitam a escolha de uma política prévia demonstraram mais eficiência. O estado inicial de ignorância, em robótica, nem sempre gera comportamento convergente, dada a possibilidade de danos físicos ao material eletrônico.

De qualquer forma, atingido o aprendizado de determinada tarefa, a memória da rede neural ganha um *preset* reutilizável no aprendizado de tarefas distintas. Um robô que obteve êxito no aprendizado do uso de determinada ferramenta, por exemplo, será mais eficiente no aprendizado do uso de uma segunda ferramenta, mesmo que pouco semelhante à primeira.

Figura 7 – Testes laboratoriais de aprendizado comparativo



Créditos: Think A/Shutterstock.

Pegar objetos é outro aspecto interessante de aplicação. Esse aprendizado é um grande desafio de robótica e não encontrou enfrentamento conveniente até o advento das redes convolucionais, que permitiram a visão computacional por extração de características da imagem. Esse método neural profundo, por si, cria o mapeamento necessário para o entendimento essencial de um objeto de forma desvinculada aos contornos e variações do meio. Os algoritmos de DRL, entretanto, ainda têm uma taxa de sucesso máxima de 96% em condições controladas, tornando o problema até o momento sem solução definitiva.

A aplicação de DRL para o aprendizado de locomoção a pé ou com pernas, por outro lado, já obteve sucesso recorrente. Nessa aplicação o problema do estado inicial de ignorância precisa ser enfrentado convenientemente de forma a evitar danos mecânicos ao protótipo. O problema foi resolvido por um treinamento inicial da rede em simuladores computacionais. Com a rede pré-setada, o algoritmo de RL é capaz de adequar o conhecimento adquirido com as interações do meio.



Nas aplicações em robótica ficam especialmente aparentes os problemas de estabilidade e memória dos algoritmos de DRL, assim o controle e a correta escolha dos parâmetros iniciais da rede se tornam o principal campo de estudos para futuras implementações.

FINALIZANDO

Concluimos nosso estudo sobre *reinforcement learning*. Certamente não foi um trajeto suave, uma vez que enfrentamos uma boa dose de matemática e álgebra vetorial. Entretanto, logramos um bom conhecimento das bases desse recente ramo de inteligência artificial. Essa base nos permitiu, nos últimos capítulos, um voo em direção ao estado da arte de IA.

Como afirmamos desde o primeiro momento, RL ainda está distante da sedimentação e estabilidade. Muito do que estudamos em breve será descartado e fará parte da história de IA. De qualquer forma, nosso estudo formou as bases necessárias para o entendimento das técnicas vindouras.

Como última recomendação voltamos a focar nos artigos técnicos referenciados, em sua maioria redigidos pelo idealizador da técnica estudada. Esses artigos nos fornecem a rara oportunidade de beber diretamente na fonte de conhecimento que produziu os algoritmos – algoritmos esses que estão sendo testados e implementados na atualidade, enfrentando, com crescente sucesso, problemas computacionais insolúveis até o momento. A leitura reiterada desses artigos, mesmo após a conclusão de seus estudos formais, será um diferencial em sua formação.

Ótimos estudos!



REFERÊNCIAS

- CHEN, W. et al. Deep reinforcement learning for Internet of Things: a comprehensive survey. **IEEE Communications Surveys & Tutorials**, v. 23, n. 3, 2021.
- ECOFFET, A. et al. First return, then explore. **Nature**, v. 590, n. 7.847, p. 580-586, 2021.
- IBARZ, J. et al. How to train your robot with deep reinforcement learning: lessons we have learned. **The International Journal of Robotics Research**, v. 40, n. 4-5, p. 698-721, 2021.
- IGEL, C.; HÜSKEN, M. Improving the Rprop learning algorithm. In: **Proceedings of the Second International ICSC Symposium on Neural Computation (NC 2000)**. ICSC Academic Press: 2000. p. 115-121.
- LECUN, Y.; KAVUKCUOGLU, K.; FARABET, C. Convolutional networks and applications in vision. In: **Circuits and Systems (ISCAS), Proceedings of 2010 IEEE International Symposium on**. 2010. p. 253–256.
- MNIH, V. et al. Human-level control through deep reinforcement learning. **Nature**, v. 518, n. 7.540, p. 529-533, 2015.
- RAVICHANDIRAN, S. **Hands-on Reinforcement Learning with Python: Master Reinforcement and Deep Reinforcement Learning Using OpenAI Gym and TensorFlow**. Packt Publishing Ltd.: 2018.
- RIEDMILLER, M. Neural Fitted Q Iteration: First Experiences with a Data Efficient Neural Reinforcement Learning Method. In: **European Conference on Machine Learning**. Springer, Berlin, Heidelberg: 2005. p. 317-328.
- SCHULMAN, J. et al. Trust region policy optimization. **PMLR – Proceedings of Machine Learning Research**, v. 37, p. 1.889-1.897, 2015.
- _____. Proximal Policy Optimization Algorithms. **arXiv preprint arXiv:1707.06347**, 2017.